

Архитектура системы (Technical Design Document)

1. Определение документа

Документ «Архитектура системы» (Technical Design Document, TDD) — это фундаментальная техническая спецификация, описывающая внутреннее устройство разрабатываемого программного продукта. Она переводит бизнес-требования (описанные в ТЗ) на язык инженерных решений, фиксируя логическую структуру приложения, потоки данных, физическое разделение на серверы/контейнеры и выбранный технологический стек.

Без этого документа проект становится зависимым от конкретного разработчика (bus factor), а добавление новых функций в будущем превращается в метод проб и ошибок.

2. Назначение документа

Данный документ используется для:

- **Синхронизации команды:** чтобы Frontend-разработчик понимал, какие данные и в каком формате ему отдаст Backend-разработчик.
- Обоснования архитектурных решений: фиксации ответов на вопросы «почему мы используем реляционную БД, а не NoSQL?» или «почему выбран этот фреймворк?».
- Визуализации неочевидных связей: графическое отображение того, как данные перемещаются между модулями, где кэшируются и как сохраняются.
- Онбординга (адаптации) новых участников: для быстрого погружения новых программистов в проект.

3. Структура документа

- **Общая схема архитектуры (System Context / Container Diagram):** графическое представление топологии системы. Должно отображать, как клиентская часть общается с серверной, где находится база данных, файловое хранилище и внешние API (например, сервисы оплаты или отправки email).
- **Стек технологий с обоснованием:** исчерпывающий перечень используемых языков программирования, фреймворков, СУБД, брокеров сообщений и утилит развертывания. Для каждой технологии пишется 1-2 предложения, почему выбрана именно она.
- **Диаграмма базы данных (ER-диаграмма):** схема физической или логической модели данных. Включает сущности (таблицы), их атрибуты (поля) с указанием типов данных, первичные (PK) и внешние (FK) ключи, а также кардинальность связей (один-к-одному, один-ко-многим, многие-ко-многим).
- **Описание ключевых программных модулей:** декомпозиция кода на независимые блоки (например, модуль авторизации, модуль биллинга, модуль уведомлений) с описанием их зоны ответственности.

- **Описание ключевых алгоритмов / бизнес-процессов:** UML-диаграммы последовательности (Sequence Diagram) или блок-схемы для наиболее сложных операций (например, алгоритм проверки доступности места при бронировании).

4. Критерии оценивания (Максимум: 10 баллов)

- [3 балла] **Архитектурная логика:** обоснованность разделения системы на микросервисы или модули монолита; адекватность стека технологий поставленной задаче (например, не использовать микросервисы для простейшего лендинга).
- [2 балла] **Проектирование данных:** соответствие ER-диаграммы правилам нормализации БД (отсутствие дублирования данных), наличие всех необходимых внешних ключей.
- [2 балла] **Описание логики:** глубина проработки алгоритмов. Проверяющий должен из описания понять, как именно программа решает самую сложную задачу из ТЗ.
- [2 балла] **Качество визуализации:** читаемость схем, наличие легенды (пояснений к стрелкам и блокам), использование общепринятых нотаций (допускается нестрогий UML, C4 model или понятные блочные схемы).
- [1 балл] **Оформление:** соблюдение структуры документа, технически грамотный язык, отсутствие стилистических ошибок.

5. Обязательные требования (Критические моменты)

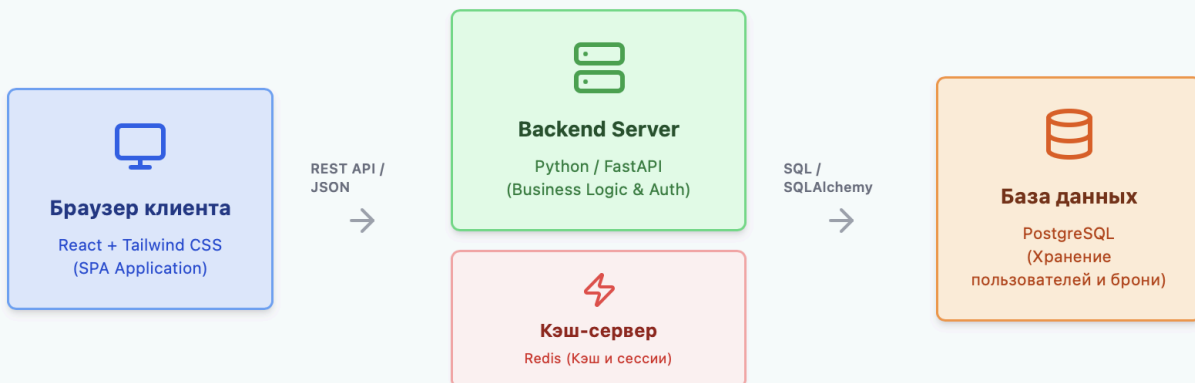
В документе должна присутствовать как минимум одна графическая схема, отражающая структуру приложения или связи в базе данных. Простой текстовый документ без визуализаций автоматически отклоняется без проверки.

Заявленный стек технологий должен строго соответствовать фактической реализации в исходном коде проекта (недопустимо описывать архитектуру на Java, если проект написан на Python).

6. Пример документа (Система управления коворкингом)

1. Схема архитектуры

1. Схема архитектуры



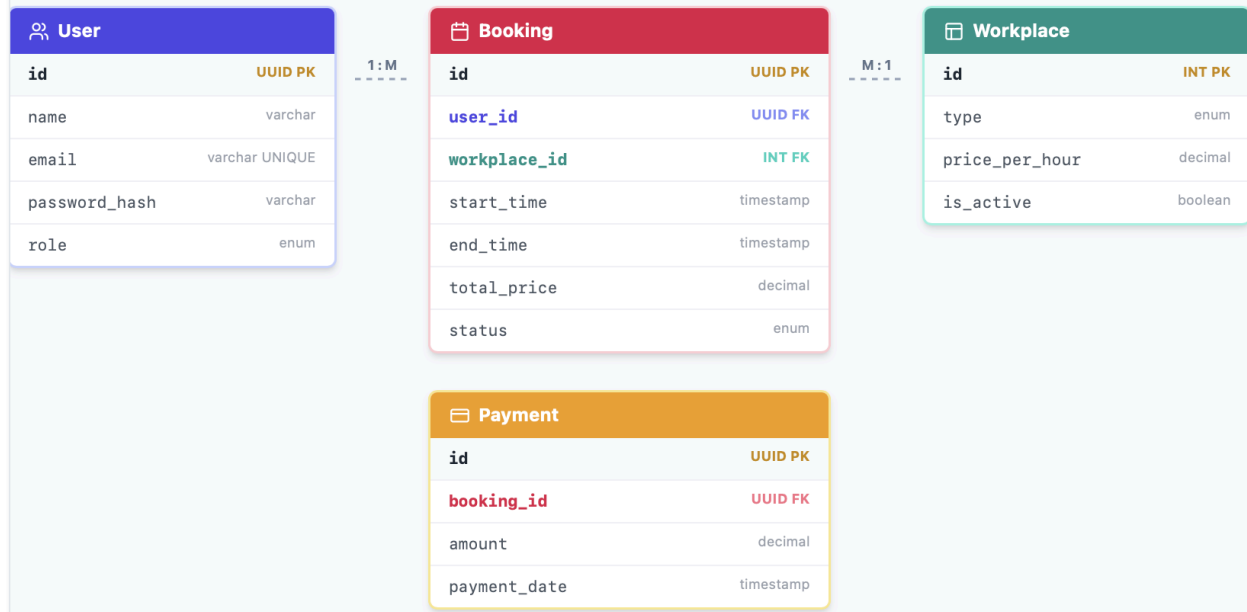
Описание схемы: Приложение построено по клиент-серверной архитектуре. Клиентская часть работает в браузере и общается с сервером исключительно через REST API с использованием JSON. Сервер обрабатывает бизнес-логику и сохраняет персистентные данные в PostgreSQL. Для ускорения загрузки каталога рабочих мест используется кэширование в Redis.

2. Стек технологий

- **Frontend:** JavaScript + React. Выбран для создания SPA (Single Page Application), что обеспечивает мгновенный отклик интерфейса без перезагрузки страниц. Для стилизации используется Tailwind CSS.
- **Backend:** Python + FastAPI. Выбран благодаря встроенной асинхронности (что важно для одновременной обработки множества запросов на бронирование) и автоматической генерации документации Swagger.
- **База данных:** PostgreSQL. Надежная реляционная СУБД, поддерживающая строгие транзакции (ACID), что критически важно для финансовых операций (исключает списание средств без подтверждения брони).
- **Кэширование:** Redis. Используется для хранения активных сессий пользователей (JWT-токенов) и кэширования списка тарифов.

3. Диаграмма базы данных

2. ER-Диаграмма БД



Текстовое описание связей:

- User (id: UUID PK, name: VARCHAR, email: VARCHAR UNIQUE, password_hash: VARCHAR, role: ENUM['client', 'admin'])
- Workplace (id: INT PK, type: ENUM['table', 'meeting_room'], price_per_hour: DECIMAL, is_active: BOOLEAN)
- Booking (id: UUID PK, user_id: UUID FK, workplace_id: INT FK, start_time: TIMESTAMP, end_time: TIMESTAMP, total_price: DECIMAL, status: ENUM['pending', 'paid', 'cancelled'])
- Payment (id: UUID PK, booking_id: UUID FK, amount: DECIMAL, payment_date: TIMESTAMP)

4. Описание модулей

- **AuthModule (Модуль аутентификации):** реализует хэширование паролей с помощью алгоритма bcrypt, генерацию и валидацию JWT-токенов доступа. Отвечает за разграничение прав (клиент/администратор).
- **BookingModule (Модуль бронирования):** ядро системы. Вычисляет стоимость на основе длительности и тарифа price_per_hour. Обеспечивает блокировку рабочего места на 15 минут в статусе pending до подтверждения оплаты.
- **NotificationModule (Модуль уведомлений):** фоновый процесс (worker), который отправляет email-письма об успешной оплате, используя SMTP-сервер.

5. Ключевые алгоритмы (Проверка доступности)

Для предотвращения ситуации, когда два пользователя одновременно бронируют один и тот же стол на одно и то же время (Race Condition), используется следующий алгоритм:

1. Пользователь инициирует бронирование стола X на время T1-T2.
2. Сервер открывает транзакцию в БД со строгим уровнем изоляции SERIALIZABLE.
3. Сервер делает SQL-запрос для поиска пересечений времени в подтвержденных

бронированиях.

4. Если пересечений нет, создается запись Booking со статусом pending. БД блокирует эту строку записи (SELECT FOR UPDATE), не давая другим транзакциям изменить ее до окончания текущей.
5. Транзакция коммитится (завершается), место успешно резервируется. Если на шаге 3 найдено пересечение, транзакция откатывается, клиент получает ошибку 409 Conflict.